Intro to Machine Learning

Major HW3

Ameer Muslih – 212209241

Mohamad Safadi - 211519467

Part 1

Q1

We need to prove the following:

$$\frac{\partial \ell_\delta(w,b{:}\,x_i,y_i)}{\partial b} = \begin{cases} (w^T x_i + b - y_i), & |w^T x_i + b - y_i| \le \delta \\ \delta * sign(w^T x_i + b - y_i), & otherwise \end{cases}$$

We'll calculate the derivative for each case separately:

- For $|w^T x_i + b - y_i| \le \delta$, we simply get:

$$\ell_\delta(w,b{:}\,x_i,y_i) = \frac{1}{2}(w^T x_i + b - y_i)^2$$

$$\frac{\partial \ell_\delta(w,b{:}\,x_i,y_i)}{\partial b} = \frac{1}{2} * 2(w^T x_i + b - y_i) = w^T x_i + b - y_i$$

- Otherwise:

$$\ell_\delta(w,b{:}\,x_i,y_i) = \delta(|w^T x_i + b - y_i|) - \frac{1}{2}\delta$$

  Since δ is independent w.r.t b we get:

$$\frac{\partial(\delta)}{\partial b} = 0$$

  We need to calculate the following derivative:

$$\frac{\partial(|w^T x_i + b - y_i|)}{\partial b}$$

  We'll split this calculation into:

  - $w^T x_i + b - y_i \ge 0$

$$|w^T x_i + b - y_i| = w^T x_i + b - y_i$$

    Since $w, x_i\ and\ y_i$ are independent w.r.t b we get:

$$\frac{\partial(w^T x_i + b - y_i)}{\partial b} = 1$$

  - $w^T x_i + b - y_i < 0$

$$|w^T x_i + b - y_i| = -(w^T x_i + b - y_i)$$

    Again for the same explanation as above, we'll get

$$\frac{\partial(-(w^T x_i + b - y_i))}{\partial b} = -1$$

To sum up we got:

$$\frac{\partial(|w^T x_i + b - y_i|)}{\partial b} = \begin{cases} 1, & w^T x_i + b - y_i \geq 0 \\ -1, & w^T x_i + b - y_i < 0 \end{cases} = sign(w^T x_i + b - y_i)$$

Thus for the "else" case we get:

$$\frac{\partial \ell_\delta(w, b: x_i, y_i)}{\partial b} = \delta * \frac{\partial(|w^T x_i + b - y_i|)}{\partial b} = \delta * sign(w^T x_i + b - y_i)$$

<u>Q2</u>

We'll define the following indicator matrices $M_1 \in \mathbb{R}^m$ as follows:

$$M_{1_i} = \begin{cases} 1, & |w^T x_i + b - y_i| \leq \delta \\ 0, & otherwise \end{cases}$$

$$\nabla_w \mathcal{L}_H(w, b) = \frac{1}{m} \sum_{i=1}^{m} \begin{cases} (w^T x_i + b - y_i) x_i, & |w^T x_i + b - y_i| \leq \delta \\ \delta * sign(w^T x_i + b - y_i) x_i, & otherwise \end{cases} =$$

$$= \frac{1}{m} X^T \left(M_1 * (Xw + b1_m - y) + \delta(|M_1 - 1_m|) * \left(sign(Xw + b1_m - y)\right)\right)$$

Note: when we write sign(M) we mean:

$$\text{sign(M)}_\text{i} = \begin{cases} 1, & M_i \geq 0 \\ -1, & otherwise \end{cases}$$

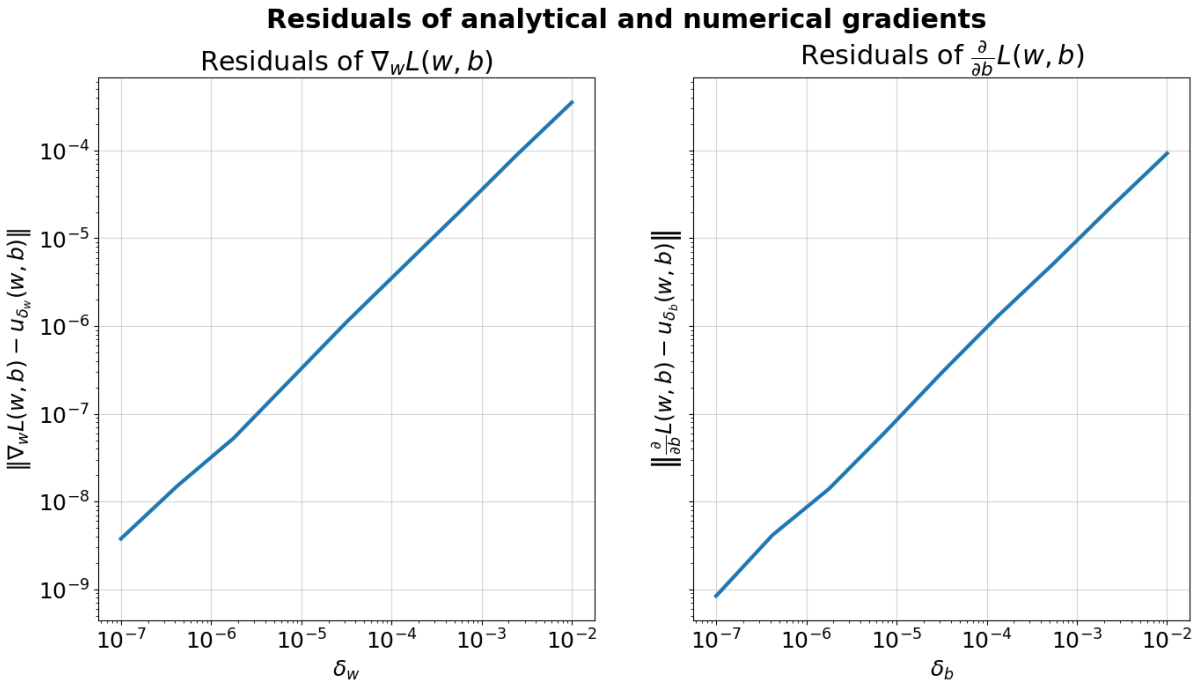
<u>Q3</u>

Psuedo-code:

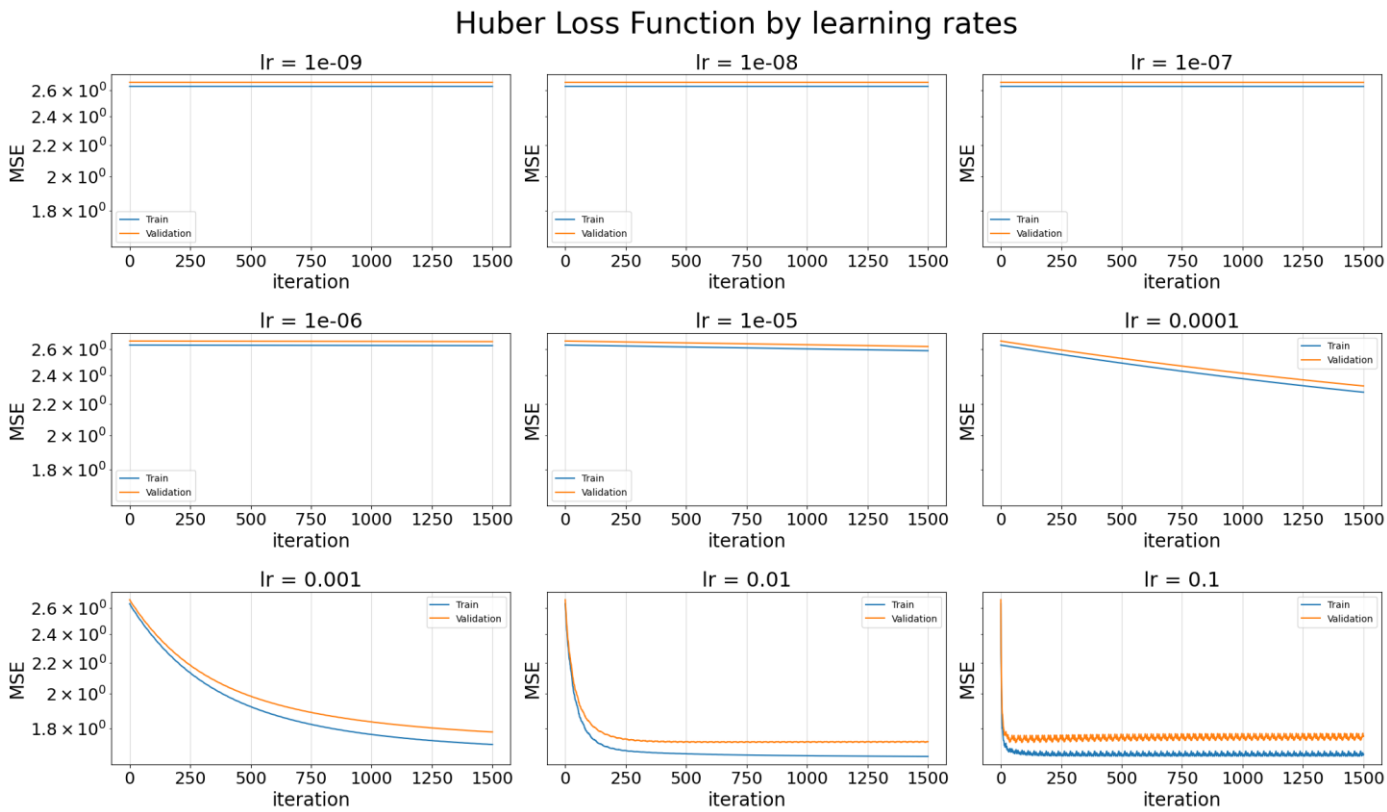
1. Split the dataset into training and validation sets.
2. Normalize the features in both the training and validation sets using statistics computed from the training set with MinMaxScaler or StandardScaler.
3. Fit a regression model (e.g., linear regression) to the training data and use MSE as our loss function.
4. Calculate the residuals for our regressor model on the training data.
5. Iterate over residual's quantiles, for each quantile Q in the range from 80 to 98:
    a. Get the Qth quantile for the residuals, representing the upper (100-Q) percentile for $|\hat{y} - y_i|$ and assign it to δ_Q.
    b. Train a regressor using the Huber loss function with δ_Q.
    c. Evaluate the regressor's MSE loss on our val set.
6. Set δ to the value corresponding to the lowest loss obtained during validation.

In this pseudo code we firstly split our data into train and validation sets in-order to later on find the best δ without bias from the training set, then we normalize the features ensuring that each feature contributes proportionally to the loss function, then we train a regression model with MSE as the loss function to get the residuals so we'd be able to find out the upper percentiles for $|\hat{y} - y_i|$ we'll use these values as our δ's and find out the best quantile for the residuals by testing the losses we get for each delta, Using the quantiles for the residuals also ensures that the Huber loss function does not give high penalties for outliers, which can improve the robustness of the model.

Q4



Q5



We can see that for very low learning rates there is no change in the loss plot.
As learning rate increases we can start seeing change as loss decreases with each iteration, this makes sense as the learning rate decides the length of each step in the gradient decent and for small enough values we might see no change in the loss at all.

The optimal learning rate in our opinion would be $lr = 0.01$ as it seems to be stable with no fluctuations (in contrast to $lr = 0.1$) and converges to a small enough value fairly quickly, we can also see that since the 500th iteration the loss stopped decreasing indicating that our loss has converged to its minimum value, and adding more iterations won't help us.

Q6

The Huber loss is particularly effective in situations where the data contains outliers, or values that are significantly different from the majority of the data. In such cases, OLS can be heavily influenced by these outliers, leading to poor performance.

The Huber loss, on the other hand, is less sensitive to outliers, as it is designed to balance the trade-off between the mean-squared error of the OLS and the absolute error of the L1 loss. This balance makes the Huber loss more robust to outliers while still providing good performance on non-outlier data.

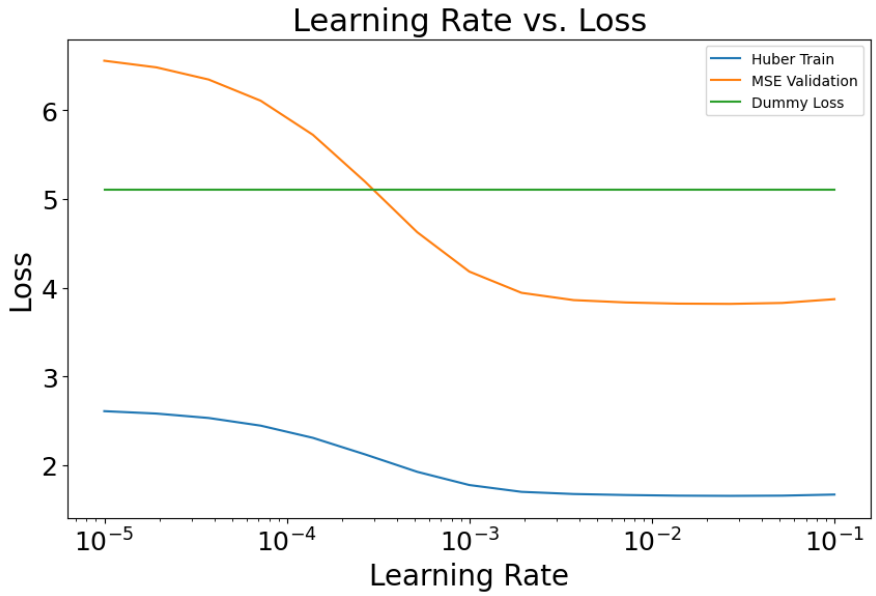
Part 2

Q7

Model	Section	Train Loss	Valid MSE
		Cross validated	
Dummy	2	5.092	5.1004

Q8

Model	Section	Train Loss	Valid MSE
		Cross validated	
Linear	2	1.6538	3.817



Best learning rate: 0.0268, Best validation MSE loss: 3.912

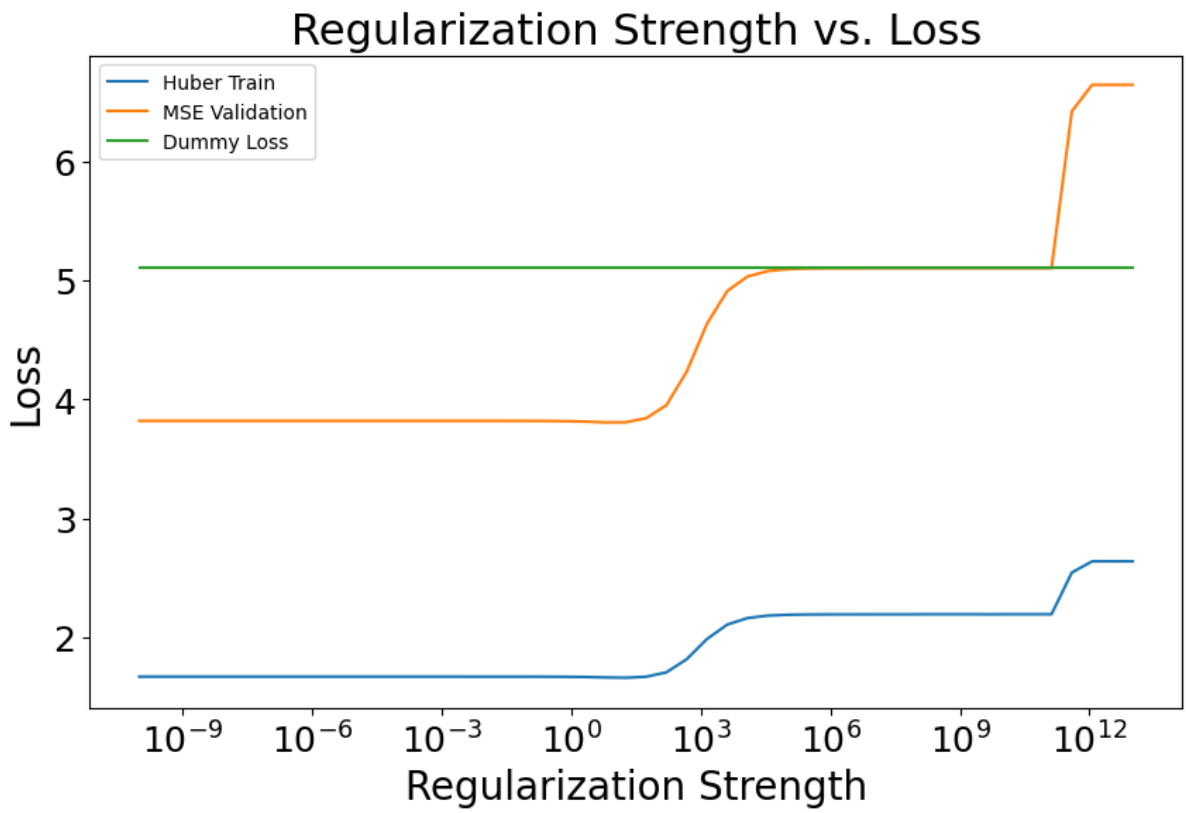
Q9

The Dummy regressor is a simple model that predicts the mean of all the labels it has seen, regardless of the input features. Therefore, it does not take into account the values of the features, and it does not require normalization of the input data. As a result, normalizing the features would not affect the performance of the Dummy regressor.

In contrast, the Custom Linear Regressor using Huber loss function takes into account the values of the features when making predictions. Normalizing the features can be beneficial for this model, as it ensures that each feature contributes proportionally to the loss function, regardless of its range. If some features have a larger range than others, not normalizing the data could lead to biased predictions. Therefore, normalization is important when using the Custom Linear Regressor with Huber loss function.

Part 3

Q10

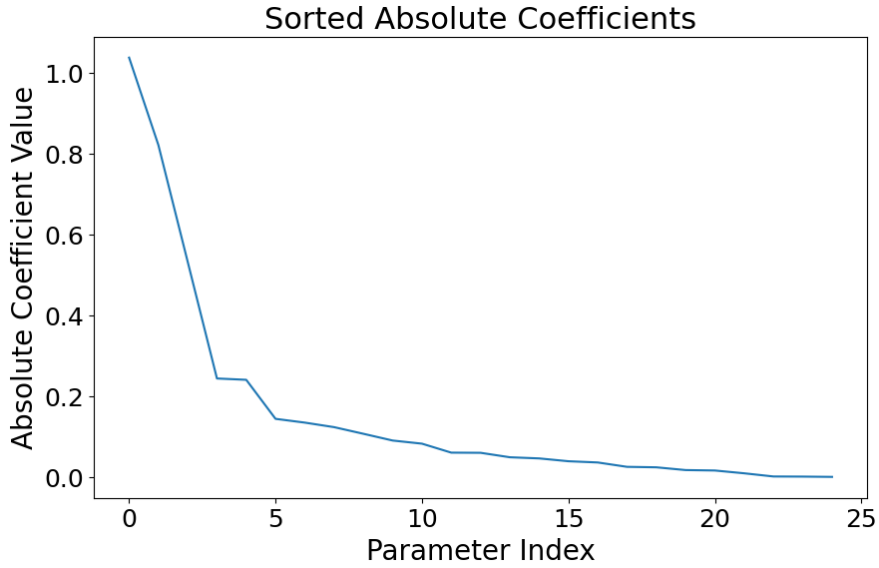


Best regularization strength: 6.25, Best validation MSE loss: 3.805

Q11

Model	Section	Train Loss	Valid MSE
		Cross validated	
Ridge Linear	3	1.661	3.805

Q12

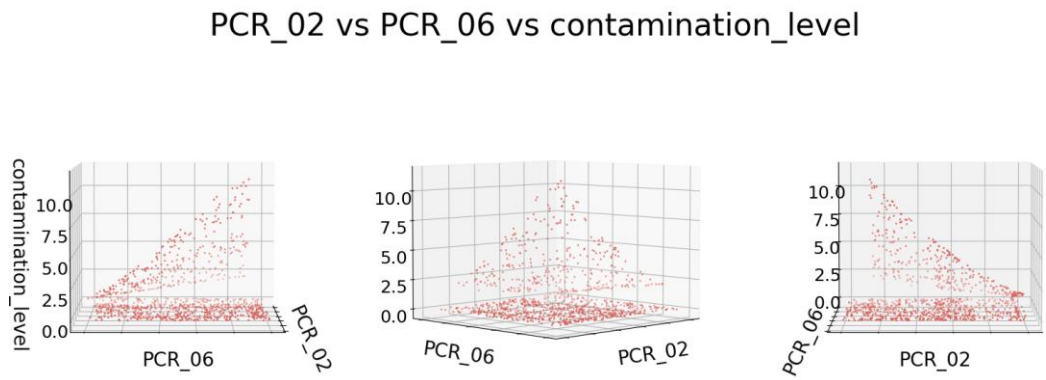


Q13

No, we'd prefer to use Lasso regularization as an option for making the model more interpretable as we've learned in the tutorials that the ℓ_1 regularization causes "variable selection", in addition to that the Lasso regularization often creates more sparse models by making some of the coefficients zero, This can help to emphasize the most important features and make the model easier to understand. Lasso regularization makes sparsity by adding an L1 penalty term to the cost function, which makes the coefficients of less important features zero.

Part 4

Q14



We can see that our data is fairly spread, where most of it lies on plane z=0 but also we've got some points that are far away from this plane that we can't call outliers, so a linear regressor on these two features won't be any good, as it will try to pass a plane that would cause a large loss due to the wide dispersion of points.

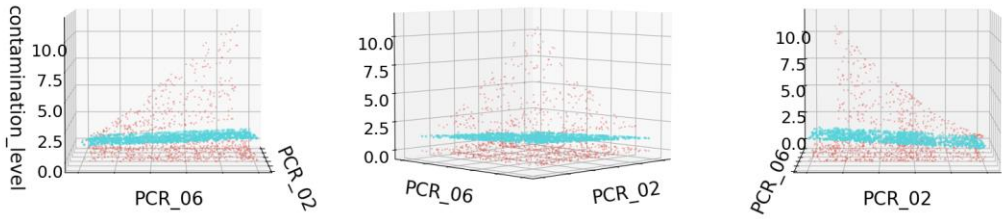
Thus, we might want to consider using more features or transform our data to higher dimension (e.g. using kernels), and we'd use a non-smooth model that could "bend" well with our data.

Q15

Code added to the submission.

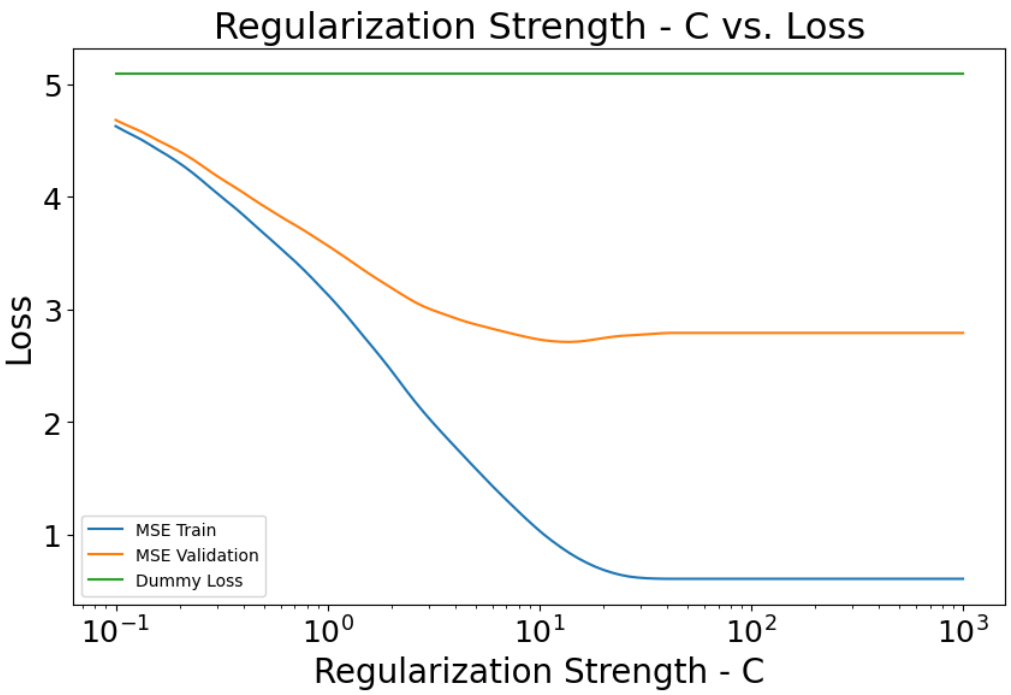
Q16

PCR_02 vs PCR_06 vs contamination_level (real and predicted(HuberRegressor))



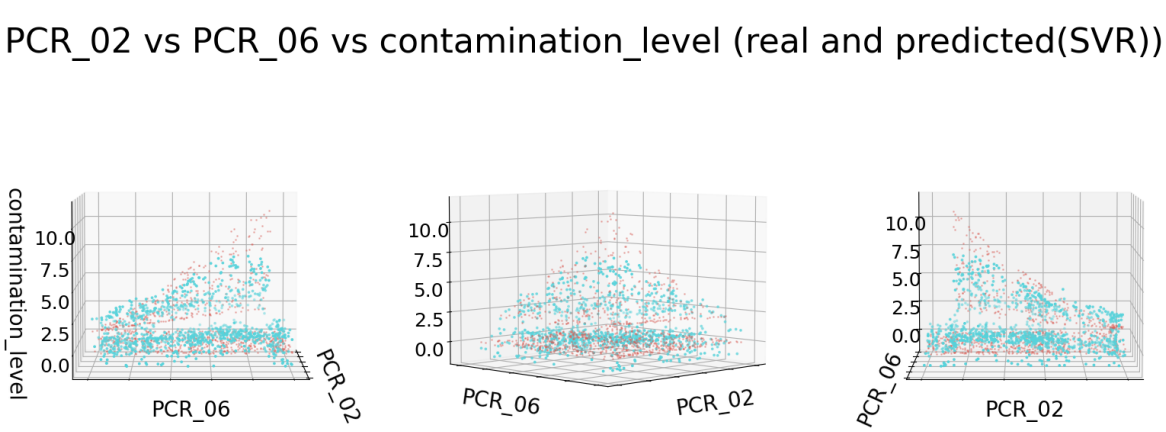
Q17

Model	Section	Train Loss	Valid MSE
		Cross validated	
SVR+Laplace	4	0.6068	2.711



Best regularization strength (C) : 13.848, Best validation MSE loss: 2.711

Q18



Q19

Comparing the plots from Q16 and Q18, we can easily see that the SVR+Laplace regressor was able to fit our data way better, and it provided us with the appropriate “split/spread” in predictions that we needed by using the Laplace kernel, on the other hand the linear HuberRegressor had bad predictions on our data as it tried to pass a plain “between” the points which caused a high train loss of 4.8968.

Q20

Model	Section	Train Loss	Valid MSE	Test MSE
		Cross validated		
Dummy	2	5.092 (MSE)	5.1004	5.89
Linear	2	1.6538 (Huber)	3.817	4.501
Ridge Linear	3	1.661 (Huber)	3.805	4.364
SVR+Laplace	4	0.6068 (MSE)	2.711	2.611

We can see that the SVR+Laplace model performed the best on our test set.